极化码算法 IR 结构搜索（Autopolar）研究记录

刘沐颉

2026.10.24

1 研究方向

本课题主要围绕极化码 SCL 译码算法的效率优化展开。在与导师及学长的交流学习中，我逐渐明确了研究思路：尝试将现有译码算法拆解为可复用的基本结构，并借助 IR，即算法中间表示，探索如何通过自动化搜索找到更适合特定约束条件的高效执行方案。

与最初将现有算法进行拼接，并根据一定的参数自动选择最优算法的思路不同，新的研究方向引入了 IR。通过将现有译码算法拆解为可复用的基本结构，再利用 IR 对不同结构及其组合方式进行统一描述和搜索，使优化不再局限于已有算法之间的选择，而是能够探索新的算法组合与执行方式，从而寻找更适合特定约束条件的高效译码方案。

现阶段的源代码和其他重要文件参见：https://github.com/LMJ-wakame/AutoPolar。

2 过程记录

2.1 搜索整理已有算法与 Baseline

主要整理了 Rate-1、单奇偶校和广义奇偶校验节点的处理方法，并建立精确搜索对照。

为此，最初的模型统一使用“路径度量、父路径编号、翻转位掩码”的顺序比较候选。这样，检查结果时不仅比较最优代价，也能比较前 L 条候选的具体内容和顺序。

这一阶段提供了后续研究的基础：每增加一种执行结构，都能够与穷举或精确基线对照，而不是仅凭候选数量下降判断算法有效。

2.2 引入 IR

早期方法与特定节点名称绑定较紧。为了能够处理更一般的节点，后续实现了通用二元线性约束表示。对于节点输出 $\boldsymbol{x}$，用

$$H\boldsymbol{x} = \mathbf{0}$$

记录合法输出必须满足的校验条件。其中 H 为二元校验矩阵，运算在二元域内进行。

若 $\boldsymbol{h}_p$ 是父路径 p 的硬判决，$\boldsymbol{e}$ 是需要翻转的位置，则检查条件可以改写为

$$H\boldsymbol{e} = H\boldsymbol{h}_p.$$

这一步使不同节点能够共用同一种约束处理接口，也支持校验关系相互重叠的情况。

实现中还压缩了重复或线性相关的校验关系。实际综合状态规模由 $2^{\mathrm{rank}(H)}$ 决定，而不直接由校验行数决定。当前通用接口要求至少一条非零校验行，无约束 Rate-1 仍通过原有入口处理。

2.3 精确搜索

在统一约束表示上，已经实现惰性精确搜索、自适应前沿搜索和综合状态 A^* 三种执行方式。

当时的主要观察是：复杂搜索方法可以减少部分候选生成，但构建辅助表和维护搜索状态也会消耗计算量。因此，后续增加了更细的操作计数，而不再仅用候选数量评价效率。

估价函数，具体地，根据平均工作量、尾部成本和回退率，比较整个节点执行与拆分子树执行的估计成本，仍需外部提供，尚未完成自动实测标定。

2.4 结构搜索

前述工作仍主要依赖预先写好的执行模板，创新空间受到限制。因此，最近一轮增加了 TrellisIR，把变量处理顺序作为可搜索结构。

同一个校验矩阵，按不同顺序处理变量，中间需要保留的综合状态数可能不同。搜索程序据此生成多个顺序，为每个顺序构建可执行网格，再比较状态规模和存储需求。这样，搜索结果会直接改变执行图中的状态和转移。

3 本阶段文献梳理记录

Fast-SSCL 及 Fast-SSCL-SPC 的工作利用列表大小限制 Rate-1 和 SPC 节点所需的比特估计次数 [1]。对本课题的启发是：候选生成范围可以由节点性质和列表大小共同决定，适合提取为带适用条件的 IR 规则。

广义快速译码研究扩展了特殊子码的识别范围，并将相关方法用于 SCL 译码 [2]。这一方向提示，处理更大节点时，需要一起考虑内部约束和候选处理开销，不能只看译码树被缩短了多少。

序列节点研究分析了 Rate-1/SPC 序列中的奇偶约束，并据此设计快速 SC 译码 [3]。高码率列表译码研究进一步讨论 SPC 和 SR1/SPC 节点的顺序与并行候选生成 [4]。这些工作使本课题更加关注用统一校验关系描述节点，以及不同生成过程之间的组合可能。

4 最新结构

新增结构用变量排列 $\pi = (\pi_0, \ldots, \pi_{n-1})$ 表示处理顺序。程序将自然顺序加入候选，再按固定随机种子产生其他排列，去重后逐个评价。

执行时主要进行三步处理：

1. 按 IR 指定的变量顺序扩展翻转位 0 和 1。
2. 检查剩余变量是否还可能完成目标校验；不能完成的状态直接删除。

3. 对同一综合状态保留前 L 条路径，最后合并不同父路径的结果。

搜索只读取校验矩阵 H，不读取随后用于正确性检查的 LLR 样本，因此本次排列选择没有使用验证输入调参。

为便于后续复查，记录当前结构评价使用的关系。设 $\boldsymbol{c}_i$ 为压缩后的校验列，处理到第 d 层时，

$$P_d = \mathrm{span}\{\boldsymbol{c}_{\pi_j} : j < d\}, \qquad S_d = \mathrm{span}\{\boldsymbol{c}_{\pi_j} : j \geq d\}.$$

P_d 表示已处理变量能够产生的综合，S_d 表示剩余变量能够补出的综合。目标为 $\boldsymbol{t}$ 时，只有满足

$$\boldsymbol{s} \in P_d, \qquad \boldsymbol{t} \oplus \boldsymbol{s} \in S_d$$

的前缀状态才需要继续保留。

对于可达目标，存活状态数可用零目标下的 $w_d = |P_d \cap S_d|$ 计算。程序依次比较

$$\left(\sum_{d=0}^{n} w_d, \quad \max_{0 \leq d \leq n} w_d, \quad \sum_{d=0}^{n} |S_d| \right).$$

三项分别对应总体状态宽度、最大宽度和后缀状态存储规模。这些量用于筛选结构，尚不是实际运行时间。

当前采用的思路是：同一父路径、同一层且具有相同综合的前缀，能够接上的合法后缀集合相同。如果一个前缀在该状态内已经排在第 L 名以后，那么给它接上任意合法后缀时，至少还有 L 个更优或按并列规则更靠前的前缀可以接上同样的后缀。因此，这条前缀不会成为全局必需的前 L 项。

这一判断建立在精确加性代价和固定并列排序上。当前代价模型为

$$M(p, \boldsymbol{e}) = M_p + \sum_i |\alpha_{p,i}| e_i,$$

其中 M_p 是父路径度量，$\alpha_{p,i}$ 是 LLR。它是原型采用的翻转代价模型，不能直接替代所有译码器的概率度量更新。浮点近并列还需要单独检查；这次新增测试采用整数 LLR。

5 实验与测试记录

本次可解释案例使用 8 个变量、2 条交错校验，矩阵为

$$H = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

校验行位掩码为 85 和 170。搜索预算为 256，随机种子为 73，去重后实际比较 255 种结构。变量下标从 0 开始，选中的顺序为

$$(1, 7, 3, 5, 6, 0, 2, 4).$$

观察指标	自然顺序	搜索所得顺序
各层存活状态数之和	26	15
最大存活状态数	4	2
后缀状态条目数之和	31	25

这次观察说明，通过 IR 搜索变量顺序确实能改变中间状态规模。总体宽度下降约 42.31%，但它只是一个案例的结构改善，不能直接写成运行速度提升 42.31%，也不能代表所有校验矩阵都能获得同样收益。

随机测试使用种子 701，生成 12 个八变量、三条非零校验行的矩阵，允许校验行线性相关。每个矩阵以预算 32 搜索结构，然后对 $L \in \{1, 2, 4, 8\}$ 分别生成 5 组输入，每组包含 2 个父路径，LLR 取 $[-5, 5]$ 内整数。

共完成 $12 \times 4 \times 5 = 240$ 组对照，均与穷举 Top-L 结果一致。该结果支持当前实现对这些有限规模输入的正确性，不等于覆盖所有实数输入或完成了形式化验证。

此前保存的 GPC4 实验包含 900 个节点案例，随机种子为 20260830，未发现结果不匹配。另有 100 帧整帧对照，条件为 $N = 16$、局部冻结 $u[0:4]$、$L = 8$，路径列表零不匹配。

已有搜索方法	平均未加权操作审计量
惰性精确搜索	1069.86
自适应前沿搜索	7684.12
综合状态 A^* 搜索	2030.72

复查后的认识是：A^* 降低了前沿方法的部分开销，但该审计量仍高于惰性精确基线。计数包含度量计算、约束检查、堆操作、子集状态操作和 DP 更新，不同操作没有按设备成本加权，也未覆盖全部运行时开销。因此，这组结果记录为成本分析和负结果，不作为实际加速证据。

这里的 900 例与 100 帧属于已有 GPC 引擎实验，不能归入新增网格结构搜索的验证成果。新增结构目前仍停留在节点级验证。

通用约束、编译入口和调度阶段曾完成 65 项测试；后续结构搜索新增的 2 项专项测试分别通过。新增测试之后尚未重新运行合并后的完整测试集，因此记录中不将二者相加后表述为“67 项完整测试全部通过”。

6 阶段性研学收获与问题记录

本阶段最大的变化，是逐渐把研究目标从“增加一种搜索方式”推进到“让 IR 搜索可执行结构”。通过统一约束表示，可以把不同节点放到同一框架中分析；通过固定排序和穷举对照，可以更清楚地判断改动是否保持了预期结果。

另一个重要认识来自成本复查。候选数下降看起来直观，但辅助表、状态维护和排序可能抵消收益。后续评价需要同时考虑结构、预计算和实际执行成本。高约束秩时，后缀集合本身仍可能指数增长，不能因为网格较窄就忽略构建开销。

目前仍有以下问题需要继续处理：

- 搜索空间主要是变量顺序，算法生成能力仍受人工规则限制。
- 新增网格搜索尚未自动接入整帧译码和实测成本调度。
- 浮点近并列下的截断安全性还需验证，现有证明义务记录不是机器证明。

- 缺少代表性码长、码率下的时间、内存和误帧率结果。
- 与成熟算法及相关结构优化方法的文献比较还不充分。

7 下一阶段拟开展的工作

扩大 IR 搜索空间，加入变量分块和局部生成器组合，观察能否生成处理流程不同的适配结构。每增加一类规则，都要先明确适用条件，再用独立输入进行结果对照。

完善成本统计，分别记录结构构建、搜索执行、排序和内存开销，并将表现合适的候选接入整帧译码器。参数选择与最终评估使用不同场景，避免只保留对方法有利的案例。

补充与 Fast-SSCL 以及网格优化、列表搜索方法的比较，确定哪些部分属于已有方法的应用，哪些部分可能形成新的研究贡献。阶段目标是给出明确的适用范围和失败条件，并获得可复现的端到端证据。

参考文献

[1] Hashemi S A, Condo C, Gross W J. Fast and Flexible Successive-Cancellation List Decoders for Polar Codes. 2017. https://arxiv.org/abs/1703.08208.

[2] Condo C, Bioglio V, Land I. Generalized Fast Decoding of Polar Codes. 2018. https://arxiv.org/abs/1804.09508.

[3] Lu Y, Zhao M M, Lei M, Zhao M J. Fast Successive-Cancellation Decoding of Polar Codes with Sequence Nodes. 2022. https://arxiv.org/abs/2204.12115.

[4] Lu Y, Zhao M M, Lei M, Zhao M J. Fast List Decoding of High-Rate Polar Codes. 2023. https://arxiv.org/abs/2311.08188.

[5] NaLv2004, “AlphaDetect,” GitHub repository. [Online]. Available: https://github.com/NaLv2004/AlphaDetect.